

고도화를 위한 LangChain

(Runnable, Memory, Streamlit)

고도화를 위한 LangChain(Runnable, Memory, Streamlit)

1. 체인의 구조를 고도화하기, Runnable
2. GPT에게 기억력을 심어주자, Memory
3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

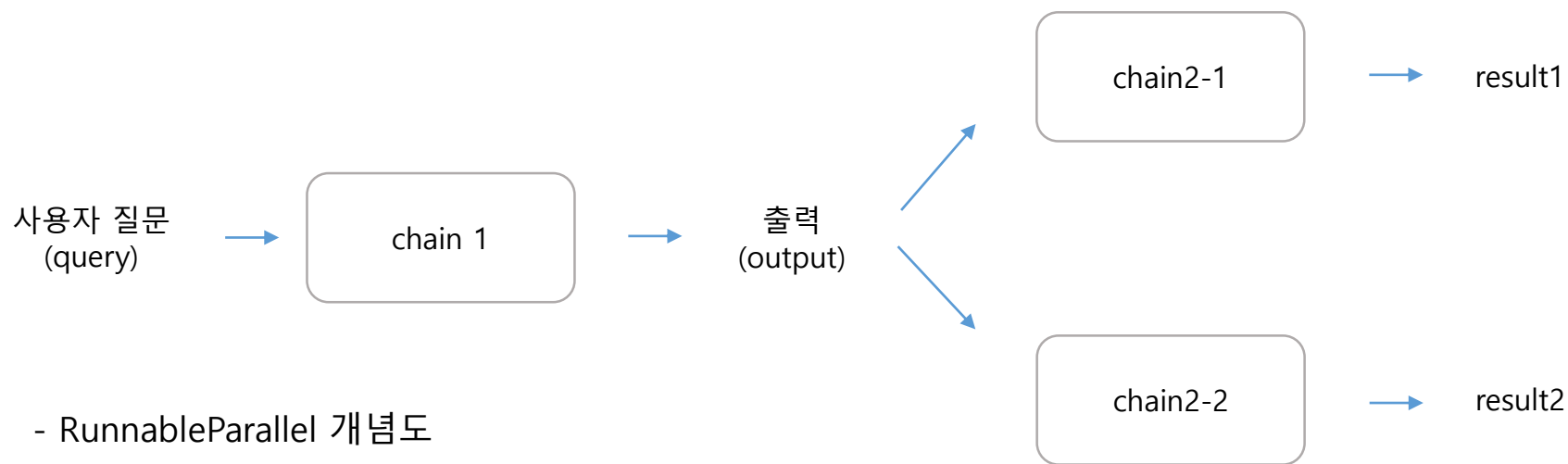
1. 체인의 구조를 고도화하기, Runnable

- 체인은 사실 랭체인 내부의 Runnable Interface를 기반으로 컴파일되어 데이터가 앞에서부터 ‘흐르듯이(Runnable하게)’ 작동함
- 프롬프트, 모델, 출력 파서를 파이프 연산자로 연결하고 invoke() 메서드로 실행하면 메서드에 전달한 딕셔너리가 체인의 요소들을 거치며(순차적으로 흐르며) 데이터가 처리됨
- 이번 장에서는 단순히 순차적으로만 실행되는 체인이 아닌, Runnable Interface를 사용해 **병렬 처리(paralleling)**, **분기 처리(routing)**, 그리고 **데이터 통과(passthrough)** 및 **확장**과 같은 고급 기능 구현
- 이를 통해 사용자 질문의 주제를 파악해 적절한 체인으로 분기 처리하고, 여러 작업을 동시에 처리하여 LLM 기반 서비스의 성능과 퀄리티를 비약적으로 향상시켜 봄
- Runnable 문법 중 특히 많이 사용되는 다음의 세가지 핵심 문법을 집중적으로 다뤄 봄
 - **RunnableParallel** : 여러 체인을 동시에 병렬적으로 실행하는 방법
 - **RunnableLambda** : 파이썬 함수를 체인에 연결하고, 이를 활용하여 조건에 따른 라우팅을 구현하는 방법
 - **RunnablePassthrough** : 데이터를 변형 없이 다음 단계로 전달하거나, 기존 데이터를 유지한 채 새로운 데이터를 추가/확장하는 방법

1. 체인의 구조를 고도화하기, Runnable

● RunnableParallel

- 2개 이상의 체인을 동시에 실행할 수 있는 방법
- 여러 작업을 병렬로 처리해 전체 처리 시간을 단축하면서, 작업 결과를 동시에 얻고 싶을 때 유용하게 활용



- ‘친구’라는 공통 주제에 대해서 각각 농담과 시를 작성하는 joke_chain과 poem_chain을 만들고, 이 두 체인을 동시에 실행시켜 봄

1. 체인의 구조를 고도화하기, Runnable

- RunnableParallel

- 개별 체인 정의

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

prompt1 = PromptTemplate.from_template("{topic}를 주제로 농담 하나 해줘")
prompt2 = PromptTemplate.from_template("{topic}를 주제로 시 하나 작성해 줘")

model = ChatOpenAI(model="gpt-5-nano")

parser = StrOutputParser()

joke_chain = prompt1 | model | parser
poem_chain = prompt2 | model | parser
```

- 병렬로 실행할 체인을 정의
 - 각 체인에 필요한 세 가지 요소 프롬프트, 모델, 출력 파서를 작성해 joke_chain과 poem_chain을 초기화

1. 체인의 구조를 고도화하기, Runnable

● RunnableParallel

- RunnableParallel 객체 생성 및 체인 할당

```
from langchain_core.runnables import RunnableParallel  
  
map_chain = RunnableParallel(joke=joke_chain, poem=poem_chain)
```

- langchain_core.runnables에서 RunnableParallel 클래스를 불러와 객체를 생성
- 이때, 병렬로 실행할 두 체인들을 전달하는데 각 체인을 특정 변수에 할당하여 전달
- RunnableParallel은 전달받은 두 체인을 동시에 실행하며 각 체인의 invoke 메서드 호출 결과를 반환
- 이때 처음 전달받은 변수명을 key로 가지는 하나의 딕셔너리를 반환
- 따라서 객체 생성 시, joke_chain을 joke 변수에, poem_chain을 poem 변수에 할당하여 생성

1. 체인의 구조를 고도화하기, Runnable

● RunnableParallel

- 체인 실행

```
map_chain.invoke({"topic": "친구"})
```

- 이렇게 생성한 map_chain으로 invoke 메서드를 호출
- 주의해야 할 점은 각 체인에서 필요한 모든 입력 변수를 딕셔너리 형태로 invoke 메서드에 전달해야 함
- 현재 두 체인 모두 {topic} 변수가 정의되어 있으니, {topic} 값을 전달하여 실행

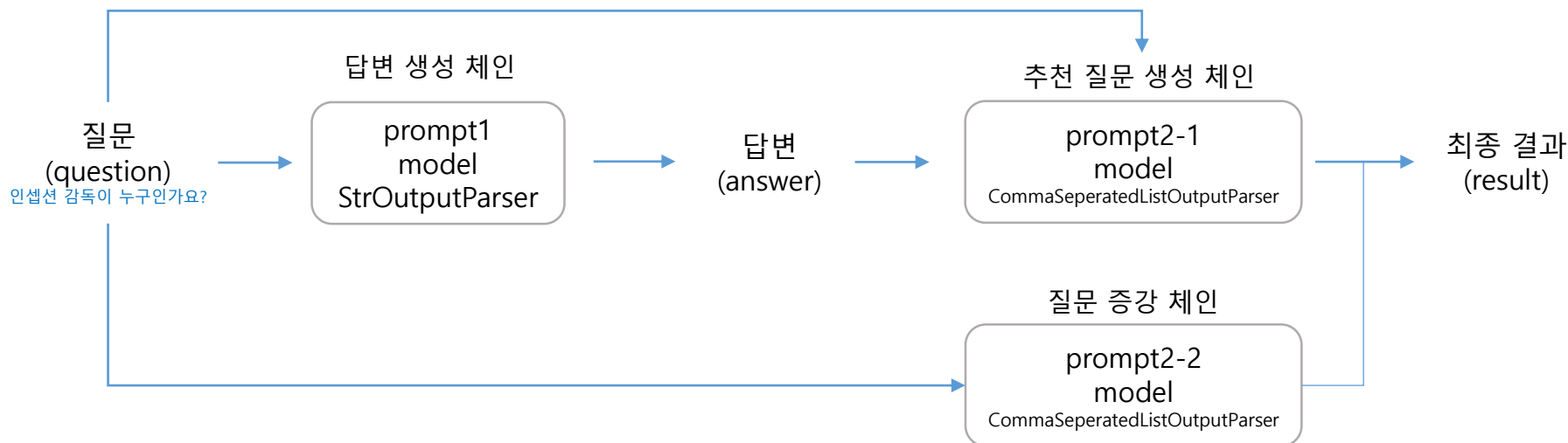
```
{'joke': '좋은 분위기의 짧은 농담 하나 드릴게요.\n\n우리 우정은 무료 와이파이야. 연결 끊길 때마다 서로의 웃음으로 다시 연결해 주지.',  
'poem': '친구야, 네 이름은 내 하루의 작은 등불\n\n해가 지고 길이 고요해질 때 네 웃음이 다시 켜진다\n\n우리가 걷던 골목은 벽에 남은 그림자처럼  
길어지고\n\n그 그림자 사이로 서로를 말없이 지켜본다\n\n점심의 빵 냄새와 수다의 물결이 한 줄의 강이 되어\n\n서로의 비밀을 나눌 때 우정은 천  
천히 자란다\n\n비 오는 날 우산 하나를 함께 쓰며\n\n빗방울이 우리를 닮아 투명하게 반사될 때\n\n우리는 말 대신 고개를 끄덕이고\n\n서로의 침묵  
에서 안식을 찾아 간다\n\n언젠가 길이 달라도 마음은 같은 방향을 본다\n\n너의 웃음이 멀리까지 닿아 나를 안심시키면\n\n나는 다시 걸을 힘을 얻는다\n\n\n그래, 고맙다, 이 작은 등불들에\n\n서로의 이름을 불러가며 남은 길을 함께 걷자'}
```

- 실행 결과로 딕셔너리 형태가 반환
- 딕셔너리의 key는 RunnableParallel에 입력한 변수명, 값은 해당 체인을 invoke 메서드를 호출했을 때의 결과물이 됨
- RunnableParallel에 입력한 변수가 key로 된 딕셔너리로 반환하기 때문에 이후 각 체인의 결과를 안정적으로 활용할 수 있음

1. 체인의 구조를 고도화하기, Runnable

● (실습) RunnableParallel 응용

- 사용자의 질문과 그에 대한 답변을 기반으로 사용자가 다음에 할 법한 '추천 질문 생성 체인'과 '사용자 질문 증강 체인'을 동시에 실행시킬 수도 있음
- 첫 번째 체인이 생성한 답변을 사용자가 확인하는 동안 사용자가 다음에 할 법한 질문들을 생성해 프롬프트 입력창에 제시함으로써 사용자 경험을 높일 수 있게 되는 것임



* RunnableParallel을 활용한 데이터 처리

1. 체인의 구조를 고도화하기, Runnable

● RunnableLambda

- 체인에 함수를 연결하고자 할 때 사용하는 방법
- 체인 중간에 특정 파이썬 함수를 삽입해 데이터를 가공하거나, 더 나아가 조건에 따라 체인의 흐름을 분기(라우팅) 시키는 기능을 구현
- 파이썬의 lambda 함수
 - 익명 함수로 파이썬에서 def 키워드 없이 간단하게 함수를 정의할 때 사용
 - “lambda 매개변수:표현식” 형태로 작성하며, 주로 재사용할 경우는 적으나 간단한 연산이 필요할 때 정의하여 사용

```
# 일반 함수
def double(x):
    return x * 2

# lambda 함수 예제
numbers = [1, 2, 3, 4]
double_numbers = list(map(lambda x: x * 2, numbers))
print(double_numbers) # [2, 4, 6, 8]
```

1. 체인의 구조를 고도화하기, Runnable

● RunnableLambda

- RunnableLambda를 활용할 수 있는 예시
 - 함수 정의 - 파이썬 기본 내장 함수 len을 바로 사용할 수도 있지만, 예시를 위해 함수 형태로 정의

```
def length_function(word):  
    return len(word)
```

- 기초 체인 정의 - 기본 프롬프트, 모델, 출력 파서 정의

```
prompt = PromptTemplate.from_template(  
    "{a} + {b}는 무엇인가요?"  
)  
model = ChatOpenAI(model="gpt-5-nano")  
output_parser = StrOutputParser()
```

1. 체인의 구조를 고도화하기, Runnable

● RunnableLambda

- RunnableLambda를 활용할 수 있는 예시
 - RunnableLambda를 통한 함수 연결 및 실행

```
from langchain_core.runnables import RunnableLambda

# Chain 구성
chain = (
    {
        "a": RunnableLambda(lambda x: length_function(x["word1"])),
        "b": RunnableLambda(lambda x: length_function(x["word2"])),
    }
    | prompt
    | model
    | output_parser
)

chain.invoke({"word1": "안녕하세요", "word2": "반가워요"})
```

- prompt에 정의한 입력 변수 {a}와 {b} 값을 얻기 위해 length_function을 연결
- RunnableLambda 모듈을 가져오고 위 코드와 같이 체인 작성
- 이렇게 체인을 작성해 invoke 메서드를 호출하면, word1과 word2의 값을 가져와 각각 length_function에 전달하고, 함수의 반환값을 “a”와 ”b”의 값으로 매핑한 새로운 딕셔너리가 완성됨

1. 체인의 구조를 고도화하기, Runnable

● RunnableLambda

- Runnable Interface 기반으로 체인이 작동한다는 것이 바로 이렇게 디렉터리가 체인의 요소들을 거치며 발전되는 방식으로 작동하는 것을 의미
- 앞의 예시는 체인의 머리에서 RunnableLambda를 사용했지만, 필요하다면 체인 중간에 삽입하여 다음 체인에 필요한 새로운 디렉터리를 구성할 수 있음
- 함수를 연결해 적절한 위치에 배치함으로써 **함수의 반환값에 따라 특정 체인으로 분기 처리하는 라우팅(routing)** 기능 역시 RunnableLambda로 구현할 수 있음
- 사용자 질문의 의도를 파악해 특정 체인으로 분기하면 LLM 서비스의 **성능과 퀄리티를 향상** 시킬 수 있음

1. 체인의 구조를 고도화하기, Runnable

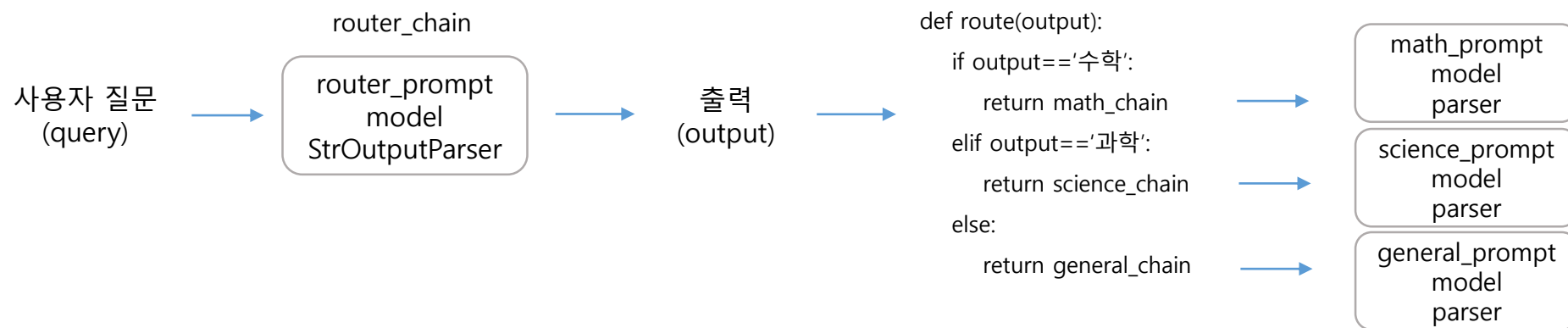
- (실습) RunnableLambda를 활용한 라우팅 구현

- LLM 모델에 한 번의 질문에 다양한 주제를 질문하면, 아마 답변의 정확도나 품질이 그리 높지는 않을 것임
- 이러한 현상을 최소화하고 싶다면, 주제별로 특화된 체인을 정의하고, 사용자의 질문을 분석해 질문의 맥락에 따라 적절한 체인으로 라우팅을 시도
- 그것만으로도 충분히 더 정확하고 전문적인 답변을 받아볼 수 있을 것

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현

- 사용자의 질문(question 또는 query)이 들어왔을 때, 해당 질문의 주제(수학, 과학, 기타)를 파악해 특정 체인으로 라우팅해 답변을 생성하는 체인 구성



* RunnableLambda를 이용한 주제별 라우팅 구현

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현
 - classifier_chain 체인 정의
 - classifier_chain은 사용자의 질문에 대한 주제를 파악해서 한 단어로 답변하는 체인

```
template = """
주어진 사용자 질문을 `수학`, `과학`, 또는 `기타` 중 하나로 분류하세요.
이외의 답변은 허용하지 않습니다.
<question>
{question}
</question>

<answer example>
수학
</answer example>
"""
```

```
classifier_prompt = PromptTemplate.from_template(template)
model = ChatOpenAI(model="gpt-5-nano")
output_parser = StrOutputParser()

classifier_chain = classifier_prompt | model | output_parser

result = classifier_chain.invoke({"question": "2+2 는 무엇인가요?"})

result
```

* 프롬프트 구성

- 가장 중요한 모델의 역할을 첫 줄에 명시(긍정적 프롬프트)
- 이후, 그 외의 답변에 대한 제어문 작성(부정적 프롬프트)
- 모델이 지시와 지시에 따라 분류해야 할 사용자의 질문을 구분하기 위해 <question>으로 구분
- 마지막으로 답변의 예시까지 작성하면 모델의 답변 형태를 최대한 제어할 수 있게 됨

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현
 - 개별 체인 생성 - 수학 체인
 - 각 주제에 특화된 math_chain, science_chain, general_chain 정의

```
# 수학 체인
math_prompt = PromptTemplate.from_template(
    """
    당신은 수학 전문가입니다.
    항상 다음과 같이 답변을 시작합니다. "피타고라스께서 말씀하시기를..."

    다음 질문에 답변하시오:

    질문: {question}
    답변:
    """
)
```


1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현
 - 개별 체인 생성 - 과학 체인
 - 각 주제에 특화된 math_chain, science_chain, general_chain 정의

```
# 과학 체인
science_prompt = PromptTemplate.from_template(
    """
    당신은 과학 전문가입니다.\
    항상 다음과 같이 답변을 시작합니다. "뉴턴께서 말씀하시기를..."

    다음 질문에 답변하시오:

    질문: {question}
    답변:
    """
)
```

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현
 - 개별 체인 생성 - 일반 체인
 - 각 주제에 특화된 math_chain, science_chain, general_chain 정의

```
# 일반 체인
general_prompt = PromptTemplate.from_template(
    """
    당신은 일반 상식 전문가입니다.\
    항상 다음과 같이 답변을 시작합니다. "부모님께서 말씀하시기를 ..."\

    다음 질문에 답변하시오:

    질문: {question}
    답변:
    """
)

math_chain = math_prompt | model | output_parser
science_chain = science_prompt | model | output_parser
general_chain = general_prompt | model | output_parser
```

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현

- route 함수 정의

- classifier_chain의 결과(주제)를 입력 받아 조건에 따라 분기 처리하는 route 함수를 정의
- 단, 이때 이 route 함수가 언제 호출될 것이며, 이때 전달되는 값이 어떤 형식인지를 생각하여 정의

```
def route(info):  
    if "수학" in info["topic"]:  
        return math_chain  
    elif "과학" in info["topic"]:  
        return science_chain  
    else:  
        return general_chain
```

- route 함수가 호출되어 분기되는 시점은 classifier_chain이 사용자의 질문을 읽고, 어떤 유형의 질문인지 파악한 뒤에야 가능
- 분류 결과는 어떤 형태로 route 함수에게 전달될까? Runnable Interface의 모든 정보 흐름은 기본적으로 딕셔너리 형태라는 점. 특정 키 값에 질문 유형의 분류 결과값이 담긴 형태로 전달될 것. 여기서는 “topic” 키에 할당되어 있을 것이라고 가정
- 조건 분기 시, ‘==’이 아닌 ‘in’을 사용한 이유는, 앞서 실행된 classifier_chain의 결과가 항상 단어라는 것을 보장할 수는 없기 때문

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현

- full Chain 연결
 - 앞에서 정의한 각종 함수와 체인들을 모두 결합하여 full_chain을 정의

```
full_chain = (  
    {"topic": classifier_chain}  
    | RunnableLambda(route)  
    | StrOutputParser()  
)  
  
full_chain.invoke({"question": "2+2는 뭔가요?"})
```

- classifier_chain의 실행 결과를 “topic”에 담을 수 있도록 첫 번째 구성 요소를 정의
- 이 구성 요소를 통해 생성된 딕셔너리({"topic": "결과값"})는 RunnableLambda를 통해 route 함수에 전달
- route 함수의 반환값(유형에 따른 최종 답변 체인)에 의해 최종 답변이 생성되고, 이를 출력 파서에게 전달
- 이 chain은 문제점 발생
 - full_chain에서 invoke 메서드를 호출할 때, 사용자의 질문을 “question”에 담아 전달
 - 그때, classifier_chain은 정상적으로 사용자의 질문을 받을 수 있지만, route 함수를 통해 실행될 각 유형별 체인은 사용자의 질문을 넘겨 받을 수 없음
 - 따라서, 실행을 시키면 KeyError 발생

1. 체인의 구조를 고도화하기, Runnable

- (실습) RunnableLambda를 활용한 라우팅 구현
 - KeyError 해결 후 최종 코드
 - 중요한 부분은 route 함수에 전달될 입력 딕셔너리를 정의하는 것

```
# 전체 체인 정의
full_chain = (
    {
        "topic": classifier_chain,
        "question": lambda x: x["question"]
    }
    | RunnableLambda(route)
    | StrOutputParser()
)

full_chain.invoke({"question": "2+2는 뭔가요?"})
```

1. 체인의 구조를 고도화하기, Runnable

● (과제) RunnableLambda를 활용한 라우팅 구현

- 사용자가 질문을 던지면, 챗봇은 이를 ****유튜브, 동물병원, 보험, 기타****로 분류한 후 적절한 체인으로 라우팅하여 응답합니다. RunnableLambda, PromptTemplate 및 체인 연결을 활용하여 동작하는 AI 시스템을 구현하시오.

****요구사항****

1. 입력 데이터

- * 사용자는 질문(question)을 입력합니다.

2. 라우터

- * 주어진 질문을 유튜브, 동물병원, 보험, 기타 중 하나로 분류해야 합니다.
- * router_chain에서 질문을 분석하고 결과를 topic 키의 값으로 반환합니다.

3. 라우팅 로직:

- * **“RunnableLambda”**를 사용하여 topic 값에 따라 적절한 체인을 선택하세요
 - * 유튜브 → youtube_chain
 - * 동물병원 → hospital_chain
 - * 보험 → insurance_chain
 - * 기타 → general_chain

4. 최종 결과 출력

- * 선택된 체인의 결과를 문자열로 반환하며, 최종 응답을 출력합니다.

1. 체인의 구조를 고도화하기, Runnable

● RunnablePassthrough

- passthrough(투과, 통과)라는 단어 뜻대로, 체인을 통과할 때 데이터를 변형 없이 다음 단계로 그대로 넘어갈 때 사용하는 문법
- 그냥 통과만 한다면 RunnablePassthrough를 왜 사용하는 걸까?

1. RunnablePassthrough를 활용하여 데이터의 흐름을 테스트하고자 할 때

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough
```

```
full_chain = (  
    {  
        "topic": classifier_chain,  
        "question": lambda x: x["question"]  
    }  
    | RunnableParallel(  
        passed = RunnablePassthrough(),  
        modified = RunnableLambda(route)  
    )  
)
```

```
full_chain.invoke({"question": "2+2는 뭔가요?"})
```

```
{'passed': {'topic': '수학', 'question': '2+2는 뭔가요?'},  
 'modified': '피타고라스께서 말씀하시기를... 2+2는 4입니다.'}
```

- RunnableParallel을 사용해 이전에 작성했던 RunnableLambda와 RunnablePassthrough를 동시에 실행해 봄
- 이 코드를 통해 route 함수에 어떤 데이터가 전달되는지 확인해 봄
- 실행 결과를 통해 알 수 있는 것은 RunnablePassthrough는 입력 받은 데이터의 원본을 완전히 유지한 상태로 답변을 내놓았다는 점
- 즉, 체인 내 다음 구성 요소에 전달될 딕셔너리 혹은 데이터의 형태를 파악하기 위해 사용

1. 체인의 구조를 고도화하기, Runnable

● RunnablePassthrough

- passthrough(투과, 통과)라는 단어 뜻대로, 체인을 통과할 때 데이터를 변형 없이 다음 단계로 그대로 넘어갈 때 사용하는 문법
- 그냥 통과만 한다면 RunnablePassthrough를 왜 사용하는 걸까?
 2. assign 메서드와 함께 사용해 데이터를 확장하기 위함
 - assign 메서드를 사용하면 기존에 전달한 딕셔너리에 새로운 키-값 쌍을 추가해 딕셔너리를 확장할 수 있음
 - 이를 통해, 이전 체인에서 얻은 데이터를 다음 체인으로 전달할 때, 원본 입력 데이터가 소실되는 문제를 해결할 수 있음
 - 앞의 RunnableLambda를 이용한 예제에서 발생한 KeyError를 RunnablePassthrough의 assign 메서드를 도입함으로써 코드를 매우 간결하게 구성할 수 있음
 - 이처럼 RunnablePassthrough.assign()을 사용하면 복잡한 체인 구성 시 데이터를 잃어버리지 않고 계속해서 확장할 수 있어, 견고하고 안정적인 파이프라인을 구축할 수 있음

```
full_chain = (  
    RunnablePassthrough.assign(topic=classifier_chain)  
    | RunnableLambda(route)  
    | StrOutputParser()  
)  
  
full_chain.invoke({"question": "2+2는 뭔가요?"})
```

'피타고라스께서 말씀하시기를... 2와 2를 더하면 4가 됩니다.'

2. GPT에게 기억력을 심어주자, Memory

- LLM은 이전 질문을 기억하지 못함
- 이전 질문을 기억하지 못하면 사용자와의 대화에서 맥락을 가질 수 없음
- 사람처럼 과거 대화 맥락을 ‘기억’하여 보다 **자연스럽고 일관성 있는 답변을 생성**할 수 있도록 돕는 것이 바로 **메모리(Memory)**
- 랭체인에서 제공하는 다양한 메모리 관리 방법들을 학습하고, 이를 통해 LLM에 기억력을 심어주는 방법 살펴봄
- 랭체인이 버전이 1.0으로 업데이트되면서 메모리를 관리하는 메인 방식을 랭그래프를 통한 방식으로 변경
- 따라서 랭그래프를 활용한 메모리 관리 방식은 랭그래프를 다룰 때 살펴보도록 하고, 여기서는 작동 원리를 이해하는 데 집중

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

1. ChatMessageHistory

- 랭체인에서 대화 맥락을 저장하고 관리하는 가장 기본적인 인터페이스
- 사용자의 질문, 모델의 답변, 시스템 지시문 등을 우리가 일일이 지정할 필요 없이 알아서 데이터를 보존하는 역할을 수행
- 사용자와 AI 간의 모든 상호작용을 순서대로 리스트 형태로 보관
- ChatMessageHistory를 사용하기 위해서는 langchain-community 패키지 필요

```
(.venv)  
$ pip install langchain-community
```

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

1. ChatMessageHistory

- ChatMessageHistory 객체 생성

```
from langchain_community.chat_message_histories import ChatMessageHistory

history = ChatMessageHistory()
print(history.messages) # []
```

- messages 속성에 대화 내역을 저장하고 이를 활용하여 이전 대화를 모델이 참고할 수 있도록 해줌

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

1. ChatMessageHistory

- 대화 내역 추가

```
history.add_user_message("안녕! 내 이름은 철수야.")  
history.add_ai_message("안녕하세요 철수님! 반갑습니다.")  
  
for message in history.messages:  
    print(message.content)
```

- 생성한 history 객체에 대화 내역을 추가할 때 주의할 점은, 프롬프트를 작성할 때와 마찬가지로 누구의 대화 내용을 의미하는지 명시해야 함
- `.add_user_message()` : 사용자의 질문을 추가하고자 할 때
- `.add_ai_message()` : 모델의 답변을 추가하고자 할 때

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

- 2. MessagesPlaceholder

- 프롬프트 템플릿에서 '이전 대화들을 넣을 자리'를 표시하는 객체
 - 이전처럼 리스트를 사용해서 대화 맥락을 주입할 수도 있지만, 이전까지 방법의 경우 대화를 갱신할 수 있는 방법이 없음
 - 사용자의 질문과 모델의 답변이 추가되었을 때, 새로 발생한 대화 맥락을 모델에게 전달할 수 있는 방법이 없음
 - 이때, MessagesPlaceholder 객체를 사용한다면 대화 맥락을 동작으로 전달할 수 있음
 - 대화 기록이 늘어나거나 줄어듦 때 특별한 작업을 하지 않아도 알아서 처리해 줌

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

2. MessagesPlaceholder

- 프롬프트 정의

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_openai import ChatOpenAI

# 기존 방식
# prompt = ChatPromptTemplate([
#     ("system", "당신은 강아지 안구 건강 관련 조언을 해주는 AI 챗봇입니다."),
#     ("human", "안녕하세요! 저희 집 강아지가 눈을 자주 긁어요"),
#     ...
# ])

# 새로운 방식
prompt = ChatPromptTemplate([
    ("system", "당신은 친절한 한국어 비서입니다."),
    MessagesPlaceholder(variable_name="chat_history"), # 이전 대화들
    ("human", "{question}")
])
```

- 리스트의 요소에 MessagesPlaceholder 객체 자체를 추가하여 프롬프트를 정의
- 이 객체에 넘겨줄 대화 맥락은 무엇인지를 나타내기 위해 variable_name을 지정해 주어야 함

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

2. MessagesPlaceholder

- 대화 내역 저장소 생성

```
from langchain_community.chat_message_histories import ChatMessageHistory

new_history = ChatMessageHistory()

new_history.add_user_message("안녕하세요. 저희 집 고양이의 이름은 치즈이고, 물고기를 좋아합니다.")
new_history.add_ai_message("안녕하세요 물고기를 좋아하는 고양이 치즈를 데리고 계시군요!")

print(len(new_history.messages)) # 2
```

- 프롬프트에서 사용할 '이전 대화들'을 저장한 대화 내역 저장소를 생성
- ChatMessageHistory 객체를 사용하여 대화 내역을 생성
- 생성한 객체를 담을 변수와 이전 프롬프트에서 지정한 변수명 'chat_history'가 같은 이름일 필요는 없음
- 대화는 한 번의 질문과 답변이 있었다고 가정함

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

2. MessagesPlaceholder

- 구성 요소 정의와 체인 생성

```
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5-nano")
output_parser = StrOutputParser()

# 체인 연결
chain = prompt | model | output_parser

# 새로운 질문하기
question = "그리고 추르도 좋아합니다."

answer = chain.invoke({
    "chat_history": new_history.messages, # 이전 대화 기록
    "question": question                  # 새 질문
})

print(answer)
```

- 체인을 구성할 모델과 출력 파서를 정의하고 체인을 연결
- invoke 메서드의 인자 딕셔너리를 구성할 때, 사용자 질문은 “question” 키에 담아서 전달
- 대화 맥락을 담은 new_history의 messages 값은 이전 프롬프트에서 정의했던 “chat_history” 키에 담아서 전달

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

2. MessagesPlaceholder

- 구성 요소 정의와 체인 생성

- 문제점: 이전 대화 맥락은 new_history 객체에 잘 담아 전달했지만, 추가로 질문한 내용에 대해서는 여전히 모델은 기억하지 못함
- 해결책: 새롭게 추가한 질문과 모델의 답변을 new_history에 추가

```
new_history.add_user_message(question)
new_history.add_ai_message(answer)
```

- 결과 확인 - 각 메시지의 타입에 보기 좋게 분류하여 출력

```
for message in new_history.messages:
    speaker = "사용자" if "human" in str(type(message)) else "AI"
    print(f"{speaker}: {message.content}")
```

2. GPT에게 기억력을 심어주자, Memory

- 대화 기록 관리하기

2. MessagesPlaceholder

- 구성 요소 정의와 체인 생성
 - 새로운 대화를 기억하고 있는지 확인

```
question = "우리 고양이가 물고기 말고 또 무엇을 좋아한다고 했었죠?"

answer = chain.invoke({
    "chat_history": new_history.messages,
    "question": question
})

answer
```

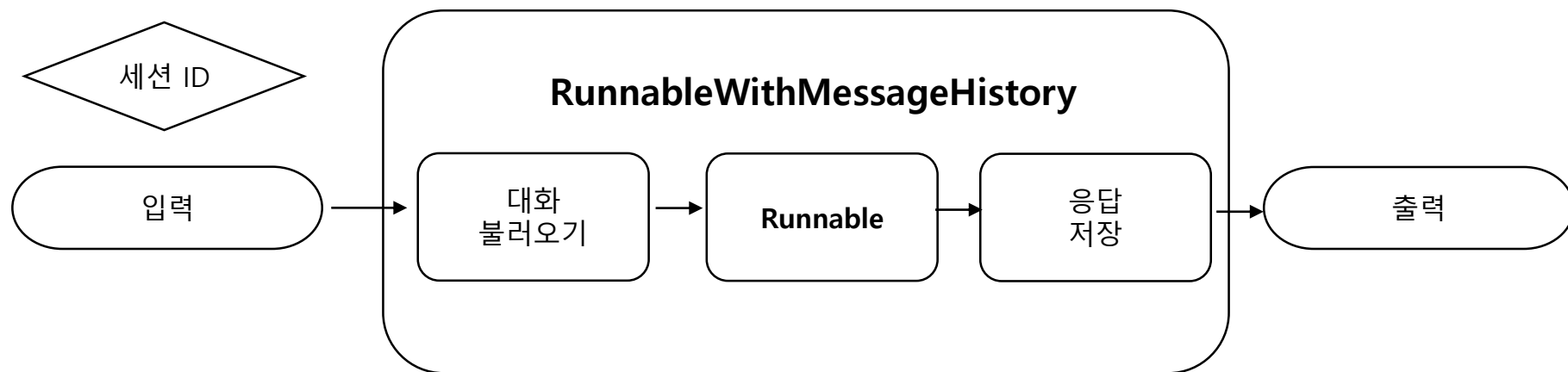
- 물론, 이렇게 추가된 질문과 답변 역시 다음 대화에서 모델이 기억할 수 있도록 하려면 추가로 저장해야 됨.
- 다소 번거롭다는 생각 -> RunnableWithMessageHistory를 살펴볼 시간

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- RunnableWithMessageHistory 기본

- RunnableWithMessageHistory의 핵심은 기존의 Runnable 객체, 쉽게 말해 invoke 메서드를 호출할 수 있는 완성된 체인에 대화 맥락 기능을 추가하는 것
 - 프롬프트, 모델, 출력 파서 순으로 진행되는 체인의 구성을 유지한 채로, 이전 대화를 불러오고 응답을 저장하는 부분을 더해주는 것



- RunnableWithMessageHistory 구성

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- RunnableWithMessageHistory 기본

- 대화 내역 저장소 생성

```
from langchain_community.chat_message_histories import ChatMessageHistory

auto_history = ChatMessageHistory()

def get_chat_history():
    return auto_history
```

- 프롬프트 정의

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate([
    ("system", "당신은 친절한 한국어 비서입니다."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}")
])
```

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- RunnableWithMessageHistory 기본
 - 구성 요소 정의와 체인 생성

```
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5-nano")
output_parser = StrOutputParser()

chain = prompt | model | output_parser
```

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- RunnableWithMessageHistory 기본
 - RunnableWithMessageHistory 객체 정의

```
from langchain_core.runnables.history import RunnableWithMessageHistory

chain_with_message = RunnableWithMessageHistory(
    chain,
    get_chat_history,          # 대화 내역 저장소를 가져오는 함수
    input_messages_key="question", # 입력 이름
    history_messages_key="history" # 기록 이름
)
```

- 앞에서 정의한 체인과 대화 내역 저장소를 하나로 묶어줄 것
- 첫 번째 인자는 반드시 Runnable 객체, 즉, 이전에 구성한 체인이어야 함
- 두 번째 인자는 대화 맥락을 불러올 수 있는 호출 가능한 객체여야 함. 이를 위해 get_chat_history 함수를 정의한 것임
- invoke 메서드를 RunnableWithMessageHistory 객체를 통해 실행할 것이므로, 기존의 체인이 가지고 있었던 입력받을 변수들을 역할에 따라 나누어 줘야 함
- 이를 위해 input_messages_key와 history_messages_key 인자를 사용해 구분 지어 줌

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- RunnableWithMessageHistory 기본
 - 연속된 대화 테스트

```
# 첫 번째 대화
question1 = "안녕! 내 직업은 영화 감독이야."
response1 = chain_with_message.invoke(
    {"question": question1},
)
print(f"사용자: {question1}")
print(f"AI: {response1}")

# 두 번째 대화
question2 = "내 직업 기억해?"
response2 = chain_with_message.invoke(
    {"question": question2},
)
print(f"사용자: {question2}")
print(f"AI: {response2}")
```

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리

- 체계적으로 세션을 관리하고, 올바른 유저 정보를 불러오기 위해서는 조금 더 복잡하고 구체적인 방안이 필요
 - ChatMessageHistory 객체를 사용자별로 식별할 수 있는 기준을 정의하고, 그 식별자가 무엇인지에 따라 서로 다른 대화 맥락을 불러오면 됨
 - 어떤 저장소를 불러올 것인지는 이전에 만들었던 `get_user_chat` 함수에서 정의
 - `RunnableWithMessageHistory` 객체가 대화 저장소를 불러오는 방법으로 저장소 객체 자체를 넘겨주는 것이 아닌, 호출 가능한 객체인 함수를 넘겨주는 형식으로 구성했던 이유가 여기에 있음

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - 대화 내역 저장소 생성

```
from langchain_community.chat_message_histories import ChatMessageHistory

user_chats = {} # 사용자별 대화를 저장하는 딕셔너리

def get_user_chat(session_id):
    if session_id not in user_chats:
        user_chats[session_id] = ChatMessageHistory()
    return user_chats[session_id]
```

- user_chats 이름의 딕셔너리를 생성하여 키 값에 따라 각각의 저장소가 생성될 수 있도록 정의
- get_user_chat 함수는 사용자를 식별할 기준 session_id 값을 인자로 넘겨 받아 사용자별 저장소를 반환해 주도록 함
- 넘겨 받은 키 값을 딕셔너리에서 찾아 아직 대화가 시작되기 전이라면(찾는 키 값이 딕셔너리에 없다면), 새로운 대화를 ChatMessageHistory를 사용하여 생성
- 이미 저장소가 만들어져 있는 상태라면, 찾아낸 저장소를 그대로 반환

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - Runnable 객체 구성

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate([
    ("system", "당신은 각 상황에 대응 가능한 박학다식한 한국어 비서입니다."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}")
])
model = ChatOpenAI(model="gpt-5-nano")
output_parser = StrOutputParser()

chain = prompt | model | output_parser

chain_with_session_id = RunnableWithMessageHistory(
    chain,
    get_user_chat,
    input_messages_key="question",
    history_messages_key="history"
)
```

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - Runnable 객체 구성
 - 체인의 기본 구성 정의
 - RunnableWithMessageHistory를 사용해 하나의 Runnable 객체로 묶어서 구성하는 부분까지 한번에 진행
 - 프롬프트, 모델, 출력 파서를 정의하고 체인을 엮은 뒤, 대화 내역 저장소와 연결하는 부분까지 모두 진행
 - 예시에서는 기존과 동일한 변수명(question, history 등)을 사용했지만, 다양한 프롬프트와 입력 변수로 변경해 보면서 전체 맥락을 익혀 보는 것을 추천

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - 대화 시작

```
# 첫 번째 사용자와 대화
question1 = "안녕하세요. 저는 선형 대수가 궁금합니다."
response1 = chain_with_session_id.invoke(
    {"question": question1},
    config={"session_id": "user1"} # 사용자1
)

print(f"사용자1: {question1}")
print(f"AI: {response1}")
```

- 우선은 한 명의 사용자를 기준으로 대화를 시작
- invoke 메서드를 사용해 연속된 대화를 진행하며, 모델이 이전 대화를 잘 기억하고 있는지 확인
- 대신, 이번에는 invoke 메서드에 config 인자를 추가함
- config 인자는 Runnable 객체를 호출할 때 사용할 구성을 전달할 수 있음. 예를 들어, 메타데이터나 병렬로 수행할 작업량을 제어하는 등이 가능
- 이번 예시에서 사용 중인 함수 get_user_chat에 넘겨줄 인자에 쓰일 값도 전달할 수 있음
- 이번 실습에서는 추가적인 설정 없이 세션ID만 user1처럼 구분하기 쉬운 문자열로만 처리하여 전달

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - 두 번째 대화 시작

```
# 두 번째 사용자와 대화
question2 = "안녕! 나는 자취생을 위한 요리가 궁금해."
response2 = chain_with_session_id.invoke(
    {"question": question2},
    config={"session_id": "user2"} # 사용자2
)

print(f"사용자2: {question2}")
print(f"AI: {response2}")
```

- 동일한 방식으로 두 번째 대화 시작
- 대신 이번에는 config에 전달하는 session_id의 값을 user2로 하여 새로운 대화가 시작되었음을 알림

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - 각 대화를 구분할 수 있는지 검증

```
# 첫 번째 사용자와 대화
common_question = "제가 뭘 물어보려고 했었죠?"
user1_response = chain_with_session_id.invoke(
    {"question": common_question},
    config={"session_id": "user1"} # 사용자1
)

print(f"사용자1: {common_question}")
print(f"AI: {user1_response}")

# 두 번째 사용자와 대화
user2_response = chain_with_session_id.invoke(
    {"question": common_question},
    config={"session_id": "user2"} # 사용자2
)

print(f"사용자2: {common_question}")
print(f"AI: {user2_response}")
```

- 검증 첫 번째는 이전 대화를 기억하고 있는지(어떤 주제를 물어보려고 했는지)
- 검증 두 번째는 각자 다른 유저에 대해서 각자 다른 답변을 내어 줄 수 있는지 확인
- 공정성을 위해 두 invoke 메서드에 질문은 하나로 통일하여 진행. 대신 config에 전달할 session_id만 서로 다르게 설정

2. GPT에게 기억력을 심어주자, Memory

- Runnable 활용

- 사용자별 대화 관리
 - 참고

```
for key, value in user_chats.items():  
    for message in value.messages:  
        speaker = key if "human" in str(type(message)) else "AI"  
        print(f"{speaker}: {message.content}")  
    print()
```

- 대화 내용을 출력해 보며 확인도 가능

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- 랭체인 기반의 챗봇 기능을 손쉽게 웹 서비스 형태로 배포하고 시각화할 수 있게 해주는 파이썬 기반의 라이브러리
- 데이터 및 머신러닝 애플리케이션을 위한 파이썬 프레임워크
- 대시보드 형태의 데이터 시각화 애플리케이션이나 챗GPT와 같은 채팅 앱을 빠르고 쉽게 개발할 수 있게 해줌
- Streamlit의 가장 큰 매력은 낮은 개발 허들 덕분에, 아이디어를 현실화하는 개념 증명 단계의 프로토타입 개발에 최적화되어 있다는 점

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 개발 환경 설정

- streamlit 애플리케이션을 로컬 환경에서 실행하기 위해서는 3.9x 이상 버전의 파이썬이 필요

- 1) 가상 환경 생성

```
(.venv)  
$ python -m venv .venv
```

- 2) 가상 환경 활성화(.venv 문구가 터미널에 추가되었는지 반드시 확인)

```
(.venv)  
$ source .venv/scripts/activate
```

- 3) 필요 패키지 설치

```
(.venv)  
$ pip install streamlit
```

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 개발 환경 설정

- streamlit 애플리케이션을 로컬 환경에서 실행하기 위해서는 3.9x 이상 버전의 파이썬이 필요

4) 'app.py' 파이썬 파일 생성하여, streamlit으로 "Hello world!" 출력

```
# app.py
import streamlit as st

st.write("Hello world!")
```

5) 실행하기

```
(.venv)
$ streamlit run app.py
```

- 서버 실행 시 이메일 주소를 묻는데, 공백으로 두어도 실행에는 문제가 없으니 Enter를 쳐서 넘어감

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 텍스트 요소

- st.title()

```
# app.py
import streamlit as st

st.title("Streamlit x LangChain")
```

- title 메서드는 텍스트를 **제목으로 지정**하여 출력하는 데 사용하는 메서드
 - 화면에 출력되는 문구의 스타일은 사용하는 브라우저(크롬, 엣지, 사파리 등)에 따라 다르게 나타날 수 있음

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 텍스트 요소

- st.header() / st.subheader()

```
# app.py
import streamlit as st

st.header("기본 문법")

st.subheader("Text element")
```

- 특정 영역의 주요 정보를 나타내거나, 추가적인 정보를 제공하고자 할 때 사용

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 텍스트 요소

- st.markdown()

```
# app.py
import streamlit as st

st.write("Hello world!")
st.markdown("Hello **world!**")
```

- 입력한 문자열에 마크다운 문법을 적용해 출력하는 메서드
 - 글자를 굵게 출력할 수 있는 마크다운 문법(** ***) 으로 “world!”를 감싸면, 텍스트가 굵게 출력되는 것 확인: st.write()와 비교

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 텍스트 요소

- st.write()

```
# app.py
import streamlit as st
import numpy as np
import pandas as pd

st.write("write를 활용한 데이터 출력")

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c']
)

st.write(chart_data)
```

- 텍스트를 화면에 출력하는 가장 기본적인 메서드
 - 텍스트뿐만 아니라 표, Matplotlib 등 다양한 데이터를 렌더링하는 기능을 제공
 - 다만, 어디까지나 단순한 형태로 렌더링하는 기능을 제공할 뿐임

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

● Streamlit

- 데이터와 차트 요소 - 다양한 데이터를 시각화하는 기능도 제공
 - st.line_chart()

```
# app.py
import streamlit as st
import numpy as np
import pandas as pd

st.write("write를 활용한 데이터 출력")

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c']
)

# st.write(chart_data)
st.line_chart(chart_data)
```

- 선형 차트(line chart)를 화면에 출력하는 메서드
- 판다스의 DataFrame 데이터를 입력하면 해당 데이터를 선형 그래프로 출력
- 추가적으로, st.area_chart(), st.bar_chart() 등 추가적으로 다양한 데이터 시각화 전용 메서드를 제공
- <https://docs.streamlit.io/develop/api-reference/data> 참조

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 레이아웃과 네비게이션 요소
 - st.columns()

```
# app.py
import streamlit as st
import numpy as np
import pandas as pd

col1, col2, col3 = st.columns([1,1,1], vertical_alignment="bottom")

with col1:
    st.header("A cat")
    st.image("https://static.streamlit.io/examples/cat.jpg")
with col2:
    st.header("A dog")
    st.image("https://static.streamlit.io/examples/dog.jpg")
with col3:
    st.header("An owl")
    st.image("https://static.streamlit.io/examples/owl.jpg")
```

... continue ...

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 레이아웃과 네비게이션 요소

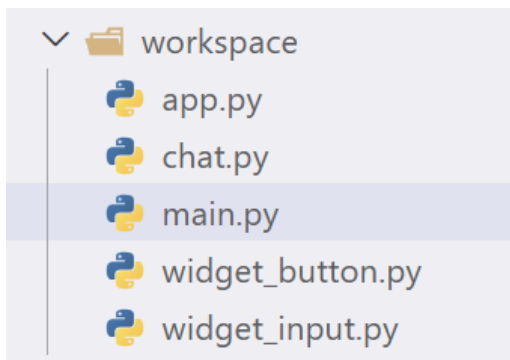
- `st.columns()`

- 화면을 여러 개의 열(column)로 분할하여 레이아웃을 구성할 때 사용
- 첫 번째 필수 인자, 리스트에 담긴 숫자(예: `st.column([1, 3, 1])`)는 가운데 열이 양 옆 열보다 세 배 넓게 3분할
- 선택 인자 `vertical_alignment`는 각 요소들 정렬 속성(“bottom”-아래로 정렬, “top”-위로 정렬, “center”-가운데 정렬)
- `columns` 메서드로 화면을 분할하면 분할된 열에 해당하는 객체(예: `col1`, `col2`, `col3`)가 반환
- 이 객체들에 `with` 구문을 사용하면 텍스트, 차트, 위젯 등 하위 요소를 담을 수 있음
- 즉, `with` 구문(`with col1 :`)을 작성하면 요소를 담을 수 있는 컨테이너가 되고, 그 안에 작성한 요소들이 컨테이너에 담김

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

● Streamlit

- 레이아웃과 내비게이션 요소
 - st.Page(), st.navigation()
 - Page 메서드를 사용하면 서비스를 구성하는 페이지를 나누어 작업할 수 있음(메서드이지만 대문자로 시작하는 점에 주의)
 - 그리고 분리된 각 페이지로 이동할 수 있도록 navigation 메서드를 사용할 수 있음
 - 실습을 위해 workspace 폴더를 만들고, 그 안에 app.py, chat.py, main.py, widget_button.py, widget_input.py파일을 만들



```
# main.py
import streamlit as st

st.title("메인 페이지")
```

```
# chat.py
import streamlit as st

st.title("챗 요소 연습")
```

```
# widget_button.py
import streamlit as st

st.title("위젯 버튼 연습")
```

```
# widget_input.py
import streamlit as st

st.title("위젯 입력창 연습")
```

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 레이아웃과 네비게이션 요소
 - st.Page(), st.navigation()

```
# app.py
import streamlit as st
import numpy as np
import pandas as pd

st.write("write를 활용한 데이터 출력")

# 페이지 생성
main_page = st.Page("main.py", title="메인 페이지", icon="0")
widget_button = st.Page("widget_button.py", title="위젯 - 버튼", icon="1")
widget_input = st.Page("widget_input.py", title="위젯 - 입력", icon="2")
chat = st.Page("chat.py", title="챗 요소", icon="3")

# 네비게이션 생성
pages = st.navigation([main_page, widget_button, widget_input, chat])

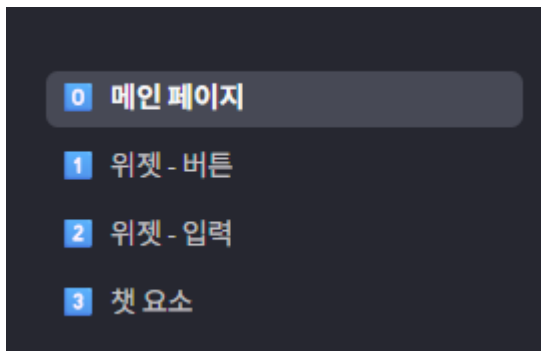
# Run
pages.run()
```

... continue ...

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

● Streamlit

- 레이아웃과 네비게이션 요소
 - `st.Page()`, `st.navigation()`
 - `st.Page`를 사용해 Page 객체 생성
 - Page 메서드를 호출할 때, 첫 번째 필수 인자로 파이썬 파일 경로 지정
 - 두 번째 필수 인자는 해당 페이지의 title을 지정할 수 있음: 네비게이션 바에 표시되는 페이지 이름이자 해당 문서의 제목 자체를 의미
 - icon 인자를 입력하면 title 옆에 아이콘이 함께 출력
 - 이렇게 생성한 Page 객체들은 navigation 메서드를 통해 관리할 수 있음
 - navigation 메서드 사용 방법
 - 화면에 렌더링하고 싶은 각 페이지 목록을 리스트로 담아서 전달
 - 반환 객체를 `pages`에 할당하고, `run` 메서드를 호출함으로써 네비게이션 요소와 현재 선택된 페이지를 화면에 출력



3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 위젯(widget) 요소 - 사용자가 데이터를 입력할 수 있는 도구
 - st.button()

```
# widget_button.py
import streamlit as st

st.title("위젯 버튼 연습")

btn = st.button("버튼을 눌러보세요")
if btn:
    st.write("버튼이 눌렀습니다!")
else:
    st.write("아직 눌러기 전...")
```

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

● Streamlit

- 위젯(widget) 요소 - 사용자가 데이터를 입력할 수 있는 도구
 - st.text_input()

```
# widget_input.py
import streamlit as st

st.title("위젯 입력창 연습")

name = st.text_input("이름", key="name")
st.title(name)
```

- 사용자로부터 텍스트를 입력받을 수 있는 기능
- text_input 메서드의 첫 번째 인자로 입력창 위에 표시될 레이블 지정
- 두 번째 인자로 key 값을 지정 -> 고유한 식별자 역할
 - 화면에 여러 개의 입력창이 있다면, 각 입력창으로부터 받은 값들을 구분할 수 있어야 하므로 입력 창마다 고유한 key를 지정
 - 따라서 key 값은 중복되지 않아야 하며, 중복될 경우 에러가 발생하니 작성 시 주의
- text_input 메서드의 반환값을 name 변수에 할당
 - name 변수는 처음에는 빈 문자열이 할당되어 있고, 사용자가 값을 입력한 뒤 엔터를 눌렀을 때, 입력한 값이 할당

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 챗 요소

- st.chat_input()

```
# chat.py
import streamlit as st

st.title('챗 요소 연습')

# 사용자 입력 받기
prompt = st.chat_input("무엇을 도와드릴까요?")

# 조건 분기를 하지 않을 경우, 화면에 None이 출력된다.
if prompt:
    st.write(prompt)
```

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 챗 요소

- st.chat_input()

- 화면 하단에 메시지를 입력할 수 있는 입력창을 생성
 - 입력창의 위치를 지정하기, 사용자가 입력한 값을 초기화하기, 입력한 값을 적절한 위치에 배치하기 등의 작업을 한 번에 제공
 - chat_input 메서드를 호출하고, 반환값을 prompt 변수에 할당 -> 사용자가 입력한 값이 할당됨
 - 이때, 파이썬 내장함수 input처럼 첫 번째 인자로 문자열을 전달하면, HTML의 placeholder 속성을 사용하여 지시문을 표시할 수 있음
 - 주의할 점은 text_input 메서드는 기본적으로 빈 문자열을 변수에 할당해 두었다가, 사용자가 값을 입력하고 엔터를 눌렀을 때 입력한 값을 전달하는 반면, chat_input 메서드는 기본적으로 None을 반환함
 - 따라서, 별도의 조건문 없이 즉시 prompt 변수에 할당된 값을 write 메서드로 출력하게 하면, 화면에 None이 출력되어 버림. 이를 방지하기 위해 조건문으로, 사용자가 값을 입력한 경우에만 문자열이 출력되도록 함

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 챗 요소

- st.chat_message()

```
# chat.py
import streamlit as st

st.title('챗 요소 연습')

# 사용자 입력 받기
prompt = st.chat_input("무엇을 도와드릴까요?")

# 조건 분기를 하지 않을 경우, 화면에 None이 출력된다.
if prompt:
    with st.chat_message("user"):
        st.markdown(prompt)
    with st.chat_message("ai"):
        st.markdown("저는 AI 입니다.")
```

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 챗 요소

- `st.chat_message()`

- 단순한 채팅창 구현이 아닌, 마치 챗GPT와 같은 화면을 구성하게 해준다는 것
 - 사용자가 입력한 프롬프트와 이에 대한 AI의 답변을 시각적으로 구분할 수 있음
 - 첫 번째 인자로, 메시지 작성자를 지정할 수 있음
 - “user” 또는 “human”을 첫 번째 인자로 넘겨주면 사용자의 메시지를 가지고 있는 컨테이너를 반환
 - 반대로, “ai” 또는 “assistant”를 인자로 넘겨줄 경우, 모델의 답변을 모은 컨테이너가 반환됨
 - 반환된 컨테이너들은 `with` 구문을 사용하여 관리할 수 있는데 `write` 메서드나 `markdown` 메서드로 출력하고자 하는 메시지를 지정하면, 적절한 아이콘과 함께 화면에 메시지가 출력
 - 현재까지는 새로운 메시지를 입력하면 대화가 누적되지 않고 새로 입력한 메시지로 업데이트 됨
 - streamlit은 사용자가 애플리케이션과 상호 작용할 때마다 파이썬 코드가 처음부터 다시 실행돼, `prompt` 변수가 초기화되기 때문

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

- Streamlit

- 세션 상태
 - st.session_state()

```
# chat.py
import streamlit as st

st.title('챗 요소 연습')

# 1) 세션 상태 초기화
if 'messages' not in st.session_state:
    st.session_state.messages = []

# 2) 사용자 입력 받기
prompt = st.chat_input("무엇을 도와드릴까요?")

# 3) 데이터 저장
if prompt:
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.session_state.messages.append({"role": "assistant", "content": "저는 AI 입니다."})

# 4) 대화 내용 표시
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
```

... continue ...

3. 랭체인 챗봇을 서비스로 구현하기, Streamlit

● Streamlit

• 세션 상태

• st.session_state()

- streamlit 애플리케이션의 중요한 특성 중 하나는 사용자가 애플리케이션과 상호작용할 때마다 파이썬 코드가 처음부터 다시 실행된다는 점
- 이로 인해 이전 대화 내역이나 변수에 할당된 값이 휘발되는 문제가 발생
- 예를 들어, 채팅 메시지를 messages라는 리스트에 저장하더라도, 사용자가 새로운 메시지를 입력할 때마다 리스트가 비워져 이전 대화가 사라짐
- 그래서 streamlit은 '상태 초기화' 문제를 해결하기 위한 session_state를 제공
- st.session_state를 활용한 대화 기록 유지 과정
 - 1) 세션 초기화 : session_state에 필요한 변수(messages)를 초기화. 이때 if "messages" not in st.session_state: 조건문을 사용하여 messages 변수가 st.session_state에 아직 없을 때만 빈 리스트(st.session_state.messages = [])로 초기화. 이렇게 하면 이후 코드가 다시 실행되더라도 이미 존재하는 messages 리스트는 초기화되지 않고 유지됨
 - 2) 프롬프트 입력받기 : chat_input 메서드를 사용하여 사용자로부터 새로운 메시지를 입력받음
 - 3) 데이터 저장 : 사용자가 입력한 메시지와 AI의 답변을 딕셔너리 형태로 구성하고, append 메서드를 사용하여 추가
 - 4) 세션에 저장된 메시지 출력 : for 반복문을 사용하여 st.session.messages 리스트에 저장된 모든 메시지를 순회하면서, chat_messages 메서드로 role을 구분하고, markdown 메서드로 content를 화면에 출력